
Open Relays in 2026

How trusted cloud workflows are modern open relays for phishing and initial access

Priyank Nigam

Microsoft Security Engineering · Breaking Ground @BSides Las Vegas

03 Aug 2026



whoami

- ▶ Priyank Nigam @ Microsoft Security Engineering
- ▶ Research focus:
 - ▶ AI Security
 - ▶ Agentic Security Automation
 - ▶ Red Team
 - ▶ App Security Assessments
 - ▶ Penetration testing
 - ▶ Secure Code Reviews
 - ▶ Bug bounties/Responsible Disclosures
- ▶ Frequent speaker at industry conferences

What we'll cover

- ▶ Exploitation of Cross-tenant email origination to arbitrary email address
- ▶ Weaponizing this for initial access via phishing
- ▶ 4 concrete examples of cross-tenant abuse of Microsoft online services
- ▶ Attachment-free red team TTPs → lower detection, guaranteed inbox placement, high click-through
- ▶ Defensive guidance for hardening email/notification pipelines

The mail server that trusts everyone (bless its heart)

- ▶ An open relay is a mail server that'll send email FROM anyone TO anyone — no ID checked at the door
- ▶ Basically a post office that happily stamps YOUR return address onto MY very sketchy ransom note
- ▶ The 1990s spammer's best friend.
- ▶ **Plot twist:** the cloud quietly rebuilt them: same energy, nicer UI, and a trusted cloud provider's return address

Case Study 1

Targeting Email Clients vs Email Protocols

Responsibly Disclosed | Remediated



Targeting Email Clients vs Email Protocols

- ▶ Modern email abuse for phishing increasingly targets client-side rendering behavior, not just SMTP / protocol flaws.
- ▶ Even when JavaScript is disabled, email clients still interpret HTML, CSS, and headers differently.

For example, attack on email protocols – SMTP header injection.

Webview/Electron Clients – Rich Rendering/Attack surface

- ▶ Chromium-based rendering engines Consistent behavior across platforms
- ▶ Support for
 - ▶ HTML
 - ▶ CSS
 - ▶ Inline resources
 - ▶ Web-like layout engines

Threat Actors love these

- ▶ HTML injection is reliably rendered
- ▶ CSS-only techniques can:
 - ▶ Hide trusted content
 - ▶ Reposition attacker content
 - ▶ Truncate or visually override warnings

Thick Clients vs Web Clients

Thick Clients (Desktop)

- ▶ Native parsing + embedded rendering engines
 - ▶ Historically inconsistent HTML/CSS handling
- ▶ Often **more permissive legacy behavior**

Web-Based Clients

- ▶ **Browser sandbox**
- ▶ More predictable rendering
- ▶ Stronger isolation, but:
 - ▶ Still vulnerable to trusted-sender HTML injection
 - ▶ Rely heavily on content sanitization upstream

Assumptions

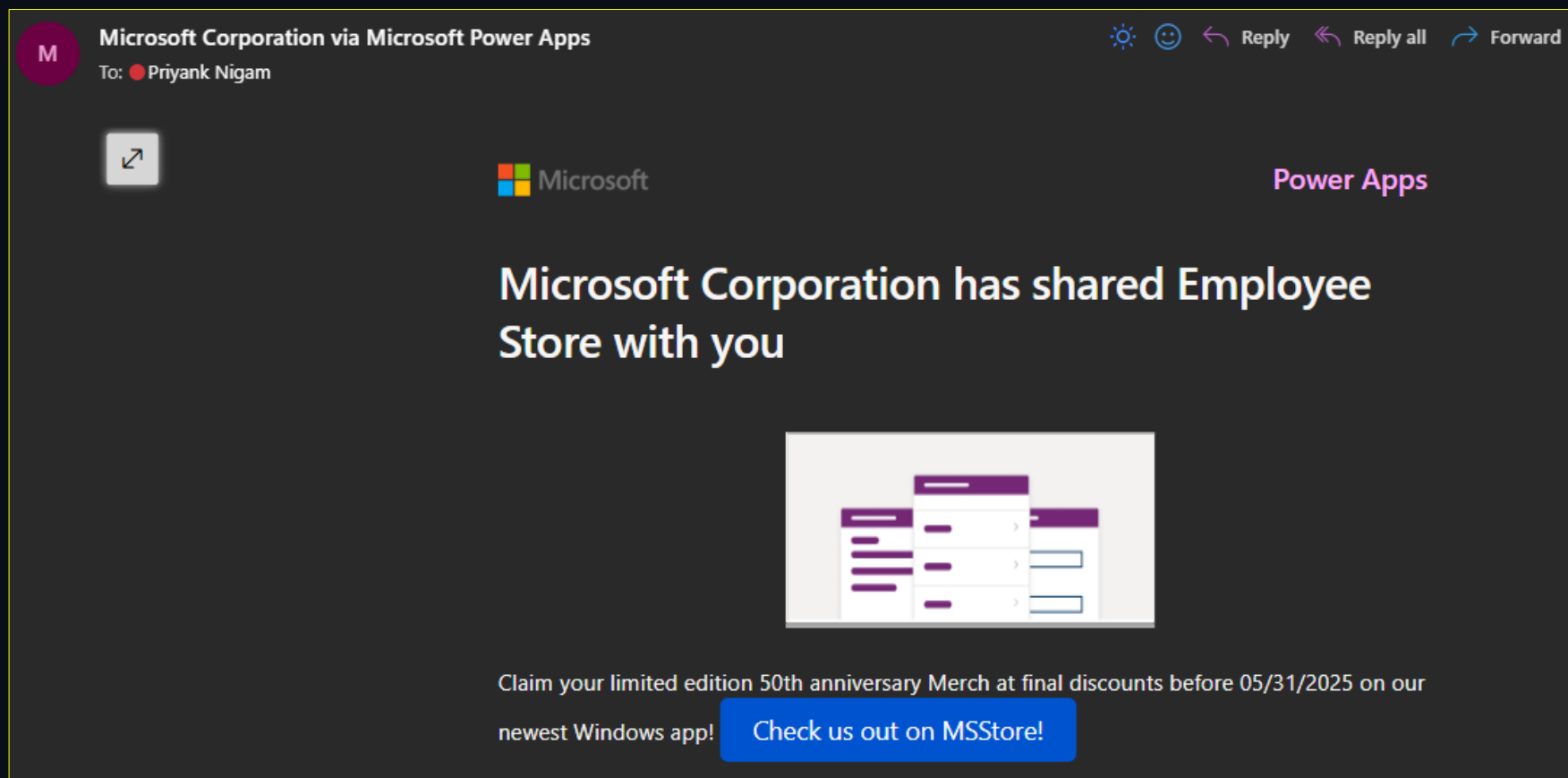
- ▶ “JavaScript is disabled, so we’re safe”
- ▶ “Email filtering will catch it”
- ▶ “Client warnings will be visible”

Reality

Exploits happen:

- ▶ Before filters
- ▶ Inside trusted workflows
- ▶ Within legitimate-looking emails

Would You Trust this Email?



What we are working with

```
{ "put": [
  {
    "properties":
    {
      "NotifyShareTargetOption": "Notify",
      "roleName": "View",
      "principal":
        { "id": "[CROSS-TENANT USER]", "type": "User" } } },
  ...omitted for brevity...
  { "emailCustomizations": { "message": "[BLAH]" } }
}
```

Only 200 chars allowed?



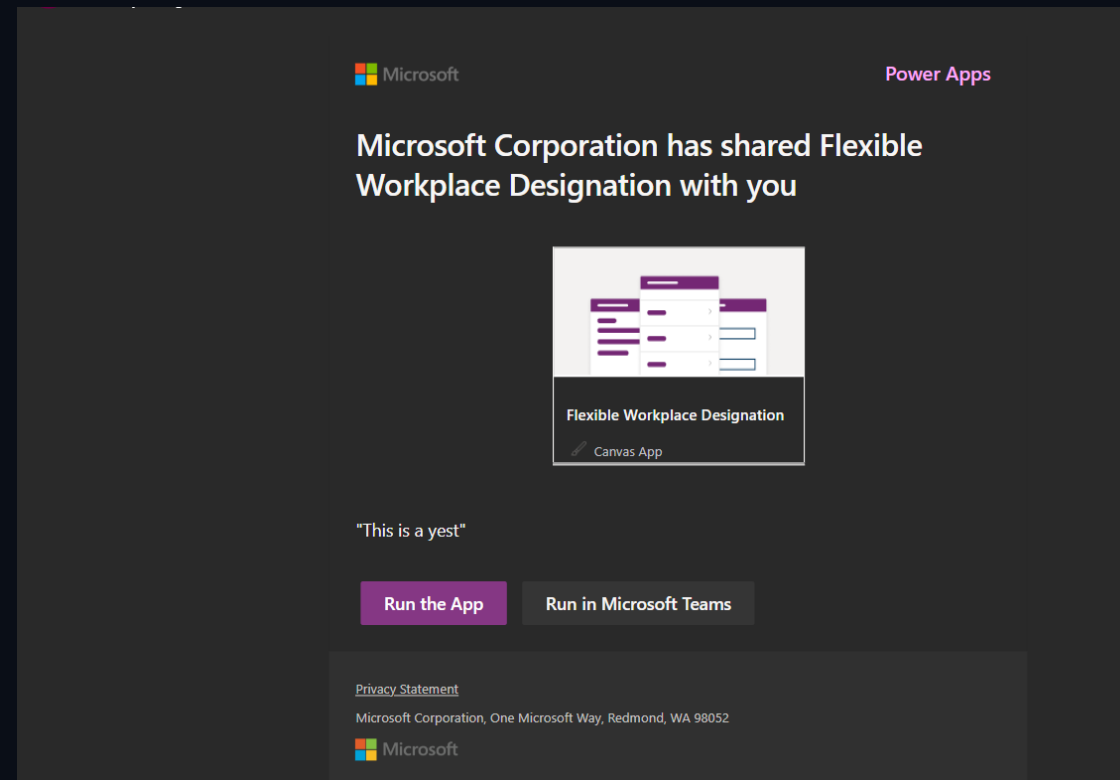
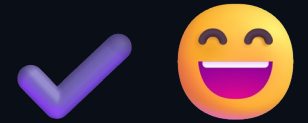
```
{
  "error": {
    "code": "InvalidModifyPermissionsCustomMessageLength",
    "message":
      "The attempted custom message is too long. Accepted character limit
      is 200. Current character limit 501."
  }
}
```

[illegible]

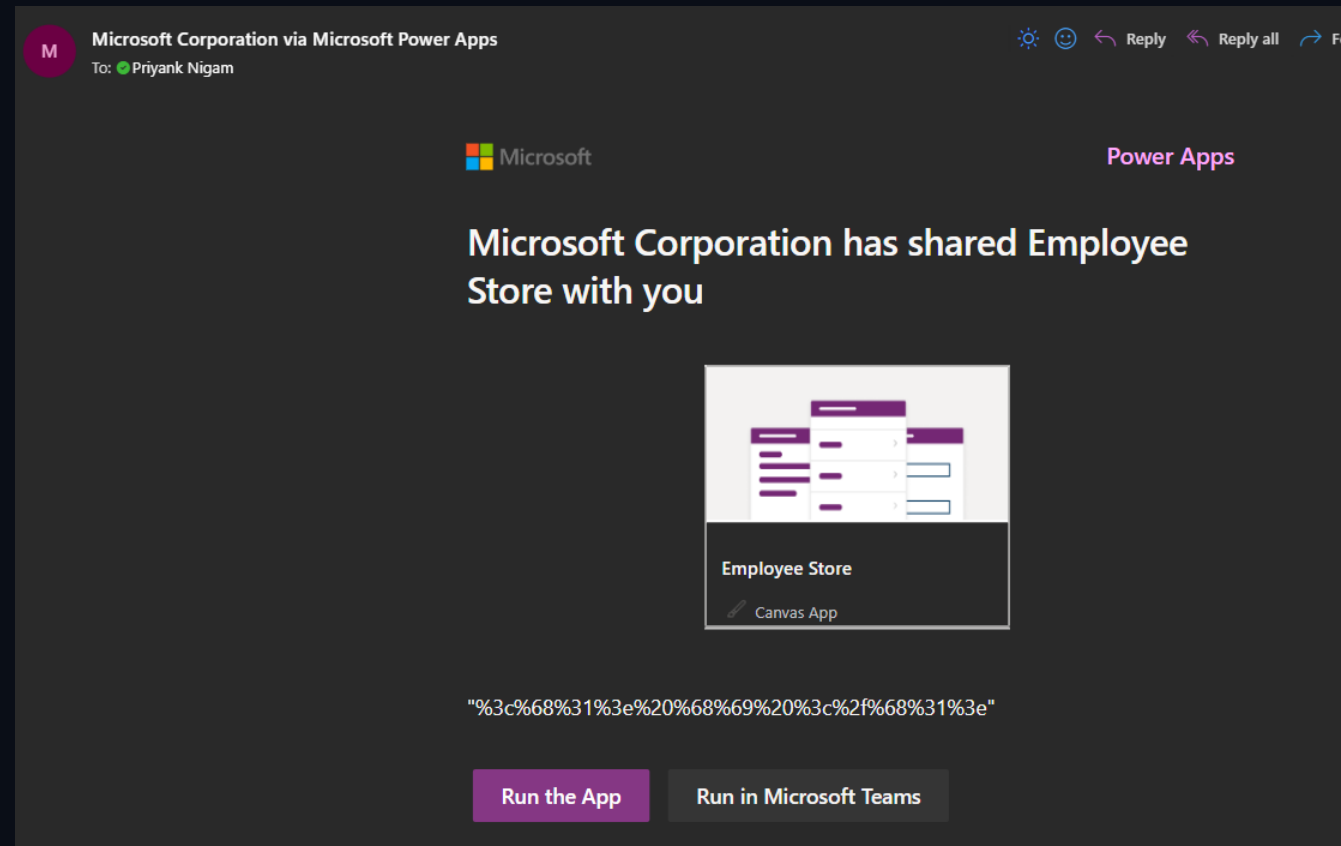
Content-Type: application/json; charset=utf-8

Open Relays in 2026: Red Team Initial Access Vectors | Priyank Nigam

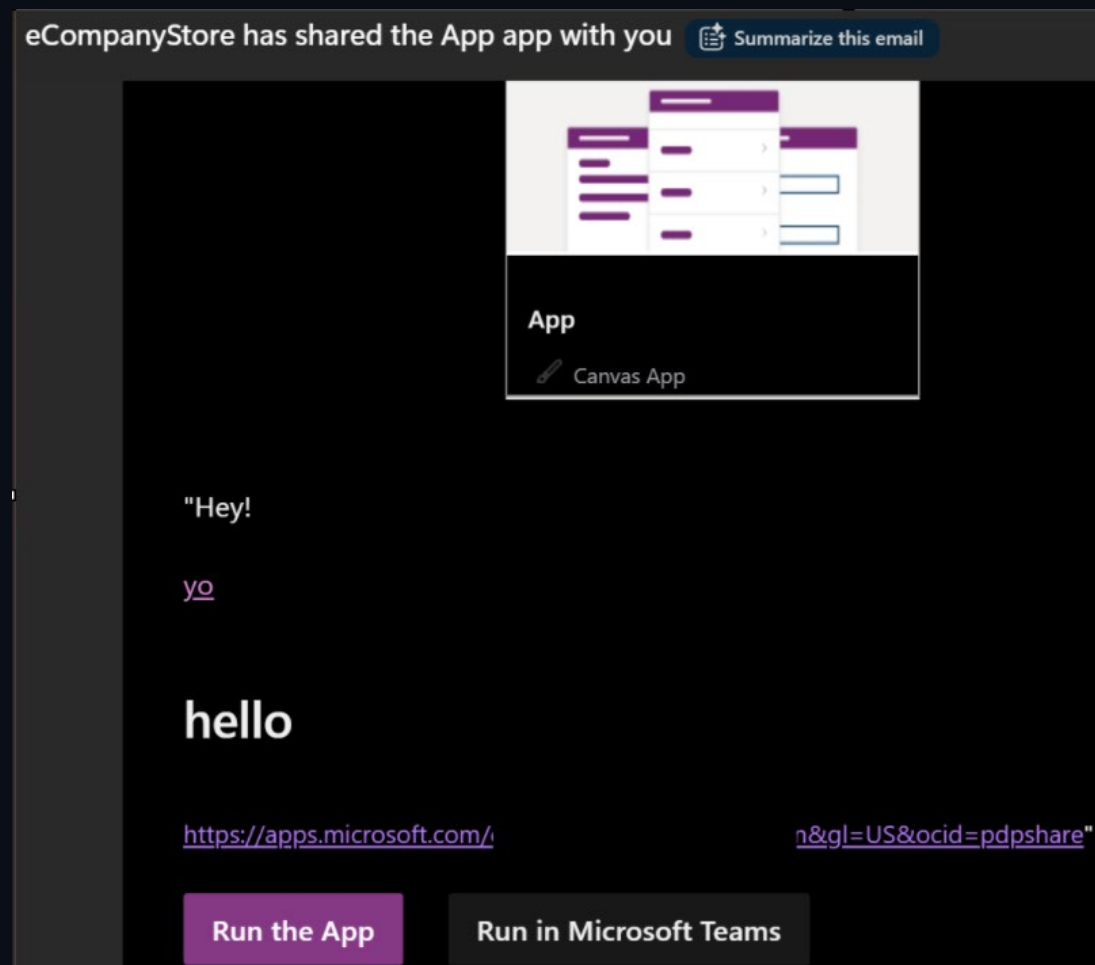
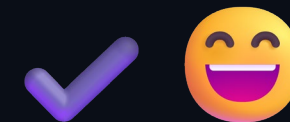
Send Normal Text



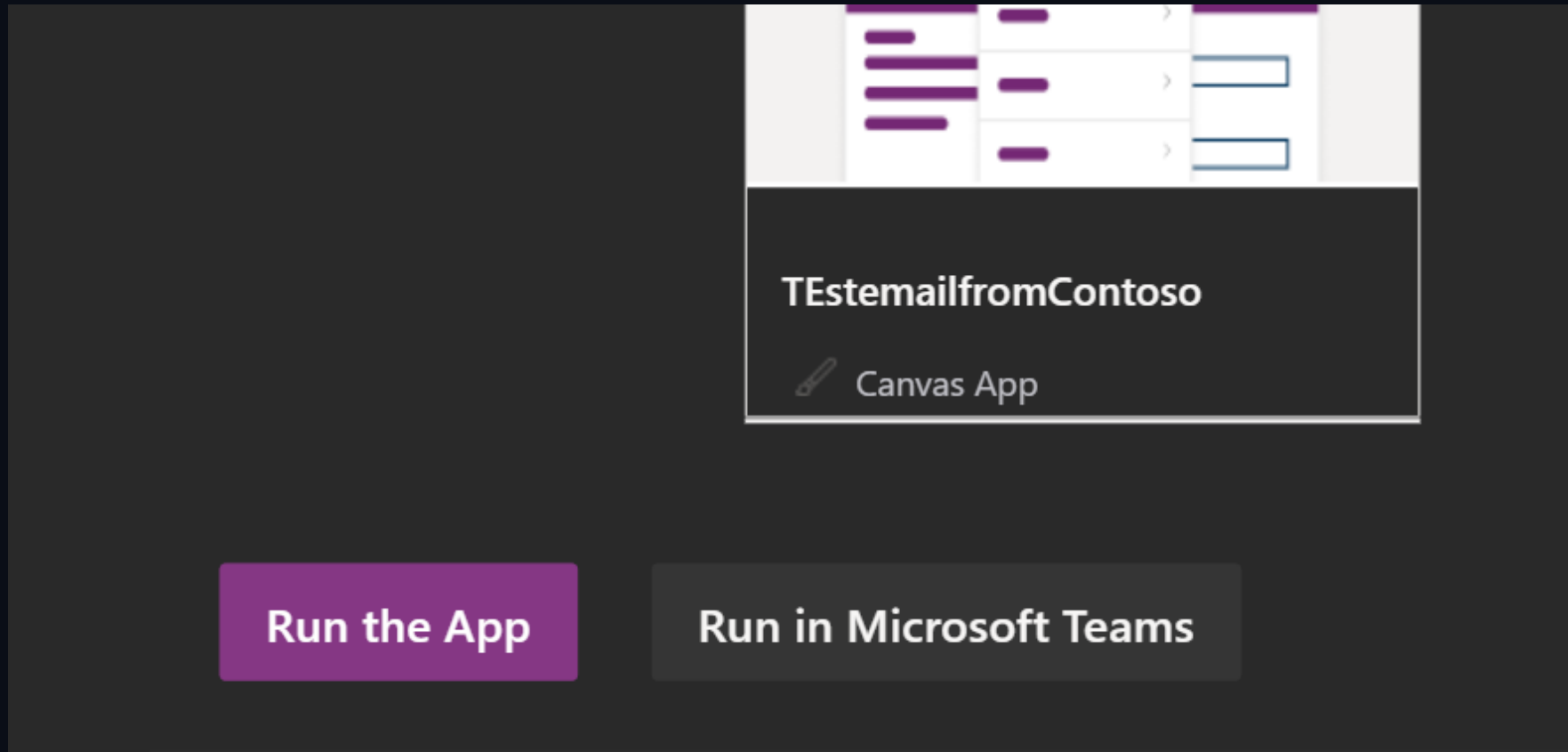
HTML Encoded Text



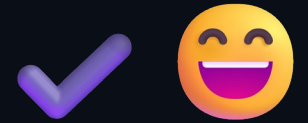
HTML Tags



Iframes?*



Href HTTP Scheme - HTTP(S)



Check out our newest app [here](#) Test.

Run the App

Run in Microsoft Teams

File/ftp/ssh/ any other protocol?

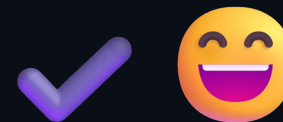


Check out our newest app here

Run the App

Run in Microsoft Teams

CSS

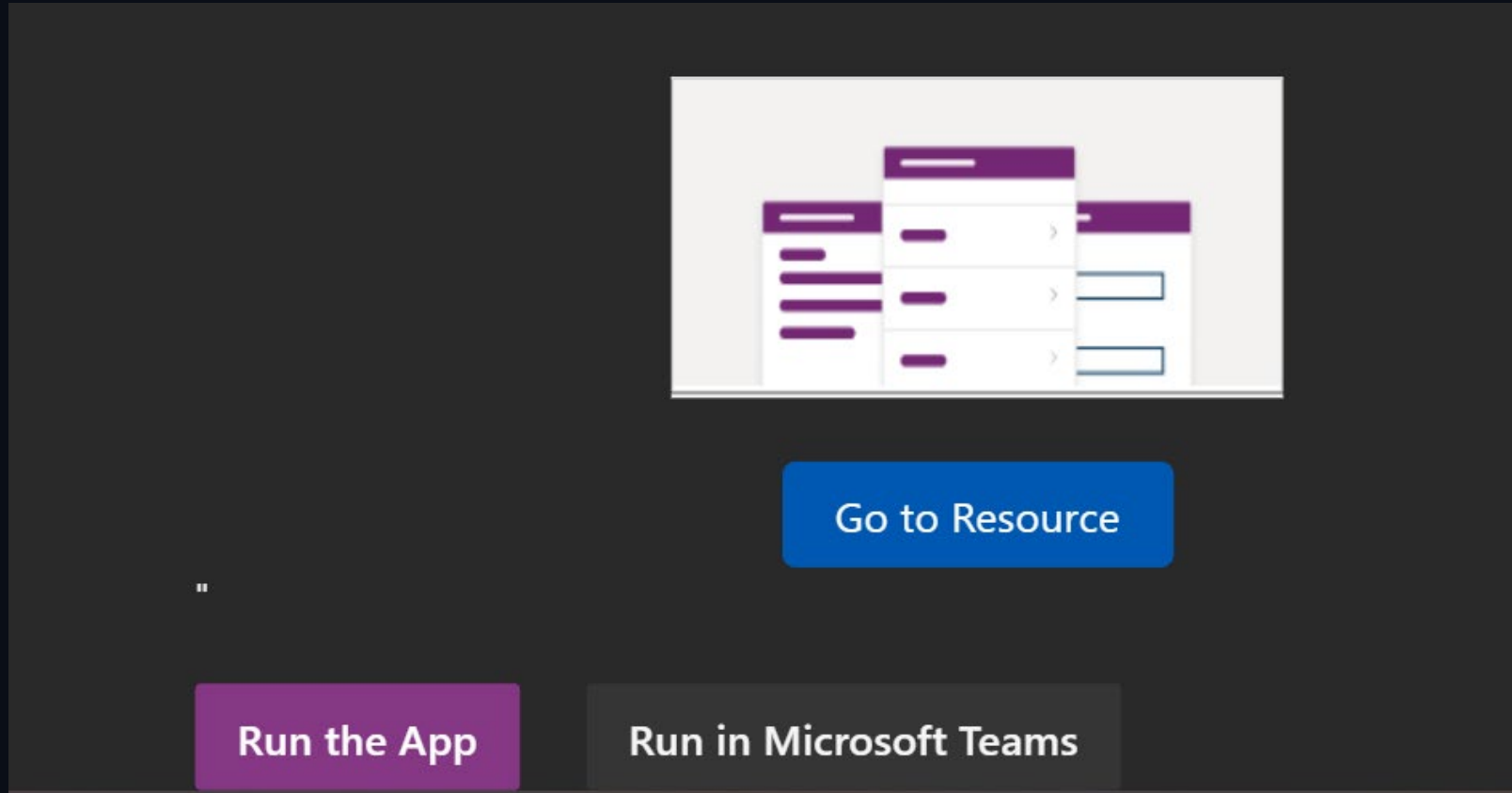
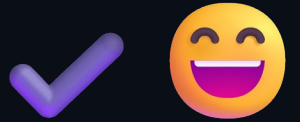


Check out our newest app [Click Here to Access](#) here

Run the App

Run in Microsoft Teams

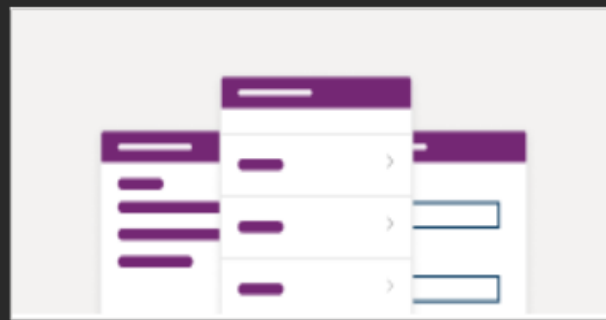
Positioning



Iframe with an allow-listed domain



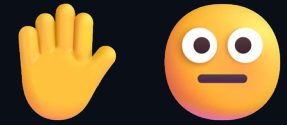
eCompanyStore has shared App with you



← Reply

→ Forward

Why was the message truncated?



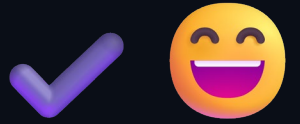
eCompanyStore has shared App with you



← Reply

→ Forward

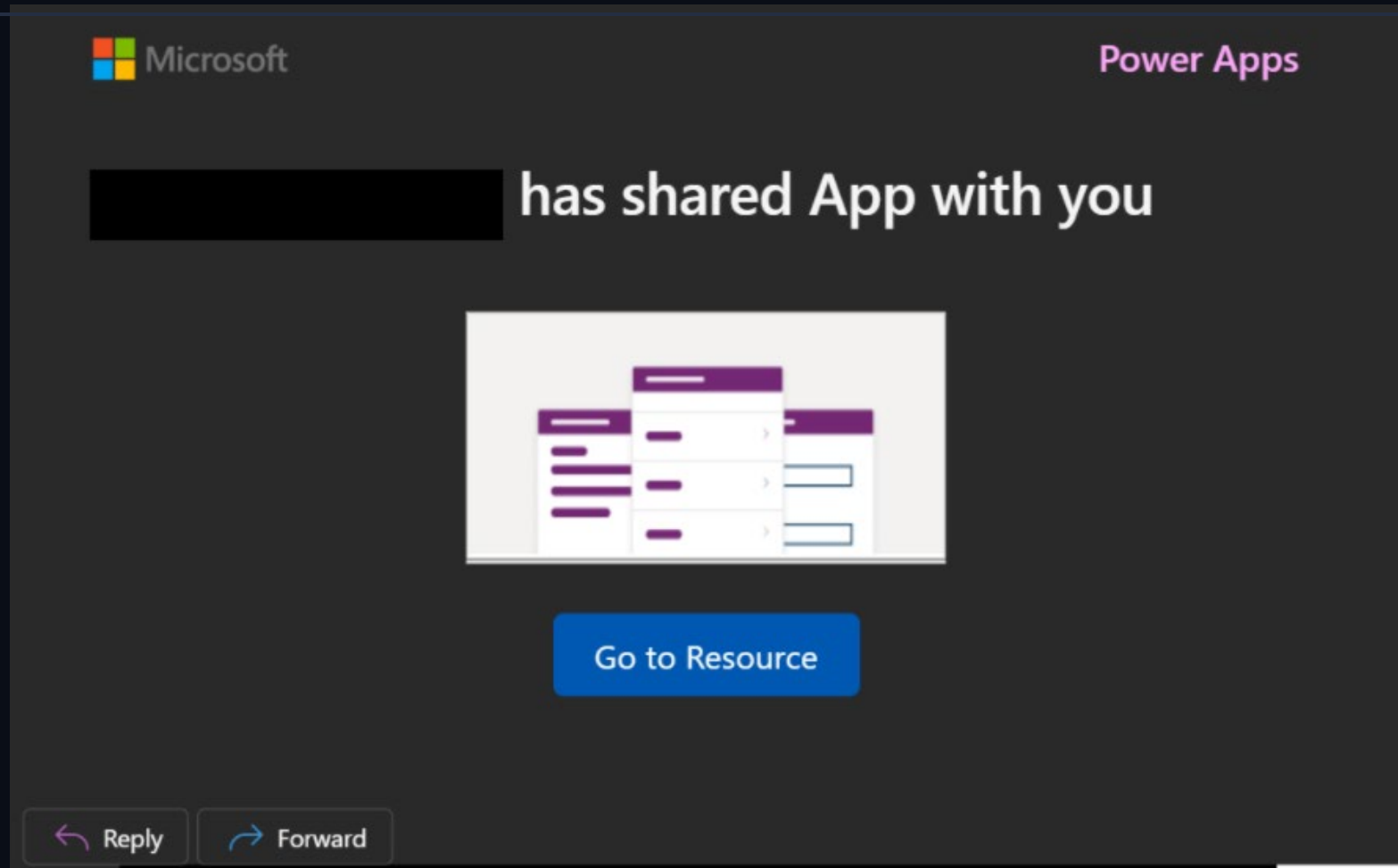
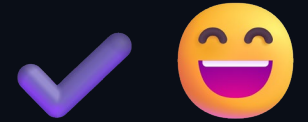
Get rid of this as well!



Run the App

Run in Microsoft Teams

Add our HTML+CSS + Truncate the server-controlled text



Final Payload



```
{"put":[
{"properties":{"NotifyShareTargetOption":"Notify
","roleName":"CanView","principal":{"id":"[REDACTED]","type":"User","tenantId":null}}}], "delete":[],
```

```
"emailCustomizations":{"message":
```

```
"</p><style>p{display:none;}</style> CHECK
THIS OUT BLAH
```

```
<a href='https://[REDACTED]' style='display:
inline-block; padding: 10px 20px; background-
color: #007bff; color: white; text-decoration:
none; border-radius: 5px; font-size: 16px;'>
```

```
Check us out HERE! </a> <span></p> <iframe
src='https://view.officeapps.live.com/op/embed.
aspx?src=' </iframe>
```

```
}}
```

```
✓ display: inline-block;
✓ padding: 10px 20px;
✓ background-color: #007bff;
!important;
✓ color: white;
✓ text-decoration: none;
✓ border-radius: 5px;
✓ font-size: 16px;
```

Fix!

```
}
1,
"delete":{
1,
"emailCustomizations":{
"message": "<p>"
}
}
}

26 Server-Timing: x-ms-igw-upstream-headers;dur=403.2,x-ms-igw-req-overhead;dur=0.5
27
28 {
  "error":{
    "code": "InvalidModifyPermissionsCustomMessageContent",
    "message": "Custom message must not contain html characters (<p>)"
  }
}
```

Share "Employee Store"



Custom message must not contain html characters (ggg<h1> fff)



PN

Priyank Nigam (g...



Add more

Case Study 2

Cross-Tenant Trusted-Sender Abuse

Responsibly Disclosed | Remediated



TL;DR

- ▶ A Microsoft cloud workflow app sends email on behalf of a privileged M365 service principal
- ▶ The 'share' feature does NOT validate that the recipient is in the same tenant — arbitrary cross-tenant recipients are accepted
- ▶ Subject + full HTML body are attacker-controlled → a highly configurable phishing vector
- ▶ Mail originates from a trusted Microsoft system domain, so it bypasses impersonation/spoofing defenses

The vulnerable workflow

- ▶ A multi-tenant cloud app (e.g., a survey app) lets users 'share' a project with collaborators
- ▶ Sharing triggers a notification email sent by a Microsoft service principal — not by the user
- ▶ Sender appears as noreply@[appname].microsoft.com
- ▶ The recipient is supplied as request parameters: an email address and a directory object id

Root cause

Missing server-side tenant validation

The backend never checks that the recipient's email address and directory object id belong to the SAME tenant as the sharer. Any arbitrary external recipient is accepted and delivered.

Attacker-controlled content

The email subject and the entire HTML body are user-controllable. HTML formatting gives full design freedom to craft a convincing phishing message.

Attack chain

- ▶ Attacker provisions their own M365 tenant and enables the app (free trial / licensing)
- ▶ Creates a project and invokes the 'share' feature
- ▶ Supplies a victim email in a DIFFERENT tenant — no server-side tenant check blocks it
- ▶ Sets a custom subject and custom HTML body = the phishing payload
- ▶ Workflow sends the mail from the trusted Microsoft domain to the victim inbox
 - PoC: mail delivered from noreply@[appname].microsoft.com, landing in the inbox as a trusted sender

Why it bypasses defenses

- ▶ Mail originates from a genuine Microsoft system domain
- ▶ Defender for Office 365 treats the sender as trusted — not impersonation, not spoofing
- ▶ SPF / DKIM / DMARC all pass because the sender really is Microsoft
- ▶ No malicious attachment → avoids attachment-based detection
- ▶ Result: near-guaranteed inbox placement and high click-through

Payload anatomy

share request – values anonymized

```
POST /api/project/share
{
  "AADObjectId": "<attacker-supplied-guid>",
  "Email":      "victim@other-tenant.example",  // not validated
  "CustomSubject": "Microsoft Shared [ATTACKER TEXT]",
  "CustomMessage": "<html> ... arbitrary HTML phishing body ... </html>"
}

// Sent as: noreply@[appname].microsoft.com (trusted sender)
```

The fix

- ▶ Validate that recipient email addresses are in the SAME tenant during sharing
- ▶ Verify BOTH the Email and the directory object id against the host tenant before sending
- ▶ Reject / block any recipient that resolves outside the sharer's tenant
- ▶ Effect: the workflow can no longer originate mail to arbitrary external recipients

Case Study 3

Anonymous access + enumerable IDs expose and let attackers approve sensitive records in a **multi-tenant** web app – triggering email notifications from trusted senders

Broken Access Control (IDOR) | Missing validation on multi-tenant app JWT

TL;DR

- ▶ An internal invoicing/approvals web app exposes an administration page that should be restricted to internal FTEs (+ onboarded suppliers)
- ▶ Any consumer MSA account can force-browse to it and read sensitive internal records
- ▶ Record identifiers are sequential → an attacker can enumerate every record in the system (IDOR)
- ▶ State-changing actions (Approve / Reject) have no authorization check — an external user could act on pending requests



MSInvoice Mail

To: ● Priyank Nigam



Microsoft Invoice

For your information, your manager has rejected your request #3.

Reason: f.

To request for another role, please submit a new request for approval.

Message sent from unmonitored alias.



Reply



Forward

The application

- ▶ A web app that manages invoice/access approval requests
- ▶ An administration page shows request details and lets approvers action them
- ▶ Intended audience: internal FTEs and a limited set of onboarded external suppliers
- ▶ Each request is addressed by a numeric requestId in the URL

Missing 'iss'/'aud' claim validation

- ▶ Multi-tenant apps accept JWT access tokens from MANY tenants — the issuer (iss / tid) alone is not a trust decision
- ▶ Correct validation must also verify the audience: the aud claim must equal THIS app's client/resource id
- ▶ If aud is not checked, a token minted for a DIFFERENT application is wrongly accepted (token / audience confusion)
- ▶ Attacker gets a valid token for any app they control, then replays it against the vulnerable multi-tenant endpoint
- ▶ Result: authentication / authorization bypass across tenants and apps

```
// decoded access token – what MUST be validated
"iss": "https://login.microsoftonline.com/<tenant>/v2.0", // issuer
"aud": "<expected-app-client-id>", // ← MUST match this app; often unchecked
"tid": "<tenant-id>", "appid": "<caller-app-id>", "scp": "..."
```

The weaknesses

1 · Broken access control

The admin page performs no real authorization. Any authenticated consumer (MSA) account — not just FTEs/suppliers — can load it and read sensitive internal records.

2 · IDOR via sequential IDs

requestId is incremental. By walking the numbers, an attacker can enumerate every record the application has ever handled.

3 · Missing authz on actions

For 'Pending Manager Approval' requests, Approve/Reject is not authorization-checked — an external user could tamper with legitimate records.

Force-browse request

```
request - host & IDs anonymized

GET /Administration/AccessRequestDetails.aspx?type=approval&requestId=<N>
Host: [invoice-app].microsoft.com
Cookie: <valid session for ANY consumer MSA account>

// increment <N> to enumerate every record (36519, 36520, 36521 ...)
```

No FTE/supplier check · no per-record ownership check · no server-side authorization

Case Study 4

Device Code phishing | Abusing OAuth 2.0 device authorization to steal tokens |
no malicious link, no password, MFA satisfied

State of the attack · 2024–2026

Sources: Microsoft (Storm-2372) · Volexity · RFC 8628

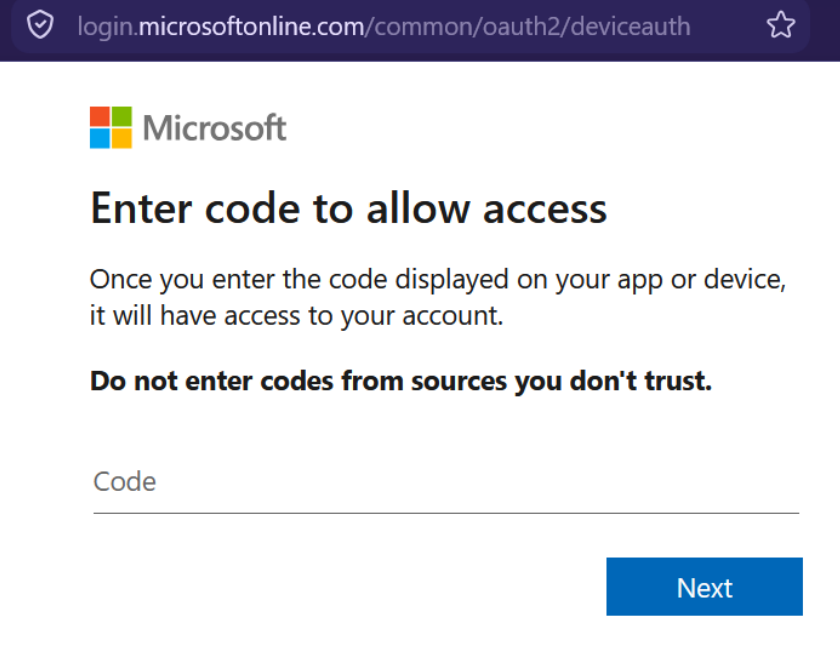
What is the device code flow?

- ▶ OAuth 2.0 Device Authorization Grant (RFC 8628)
- ▶ Designed for input-constrained devices: smart TVs, IoT, printers, CLIs
- ▶ The device shows a short user_code; the human authorizes it on a second device with a full browser
- ▶ Microsoft entry points: microsoft.com/devicelogin, aka.ms/devicelogin, login.microsoftonline.com/common/oauth2/deviceauth
- ▶ A legitimate, widely-enabled feature — which is exactly why it is abusable

The legitimate flow

OAuth 2.0 device authorization grant (RFC 8628)

1. Device → IdP: `POST /devicecode (client_id, scope)`
 2. IdP → Device: { `device_code`, `user_code`, `verification_uri`,
`expires_in: 900`, `interval: 5` }
 3. Human opens `verification_uri`, signs in, enters `user_code`, consents
 4. Device polls: `POST /token (grant_type=device_code, device_code)`
→ pending... pending... then:
 5. IdP → Device: { `access_token`, `refresh_token`, `id_token` }
- // Note: whoever polls the token endpoint receives the tokens



The screenshot shows a web browser window with the address bar displaying `login.microsoftonline.com/common/oauth2/deviceauth`. The page features the Microsoft logo at the top. Below it, the heading "Enter code to allow access" is followed by a paragraph: "Once you enter the code displayed on your app or device, it will have access to your account." A warning in bold states: "Do not enter codes from sources you don't trust." There is a text input field labeled "Code" and a blue "Next" button at the bottom right.

The attack

- ▶ Attacker (not a real device) starts a device code flow and gets a user_code + device_code
- ▶ Attacker socially engineers the victim to enter that user_code on the REAL Microsoft sign-in page
- ▶ Victim authenticates normally — password + MFA are satisfied on genuine infrastructure
- ▶ Attacker polls the token endpoint and receives the victim's access + refresh tokens
- ▶ Persistent access via refresh tokens; Graph API used to read/exfiltrate mail and files

Attack flow, step by step

who-does-what across the two devices

attacker

Microsoft IdP

victim

| -- /devicecode ---> |

| <- user_code -----|

| ----- lure w/ user_code (Teams/Signal) --> |

| | <- sign in + MFA --- |

| | <- enter user_code - |

| -- poll /token ---> |

| <- access + refresh token -----|

| == Graph API: read/exfiltrate mail, files =>

login.microsoftonline.com/common/oauth2/deviceauth

Microsoft

Enter code to allow access

Once you enter the code displayed on your app or device, it will have access to your account.

Do not enter codes from sources you don't trust.

Code

Next

Why it evades defenses

No malicious artifact

The only link is to genuine Microsoft infrastructure. No attachment, no lookalike domain — email filters and users see nothing 'malicious'.

MFA is satisfied

The victim completes real MFA on the real page. Standard MFA does NOT stop this — the token is issued legitimately, then stolen.

Benign sign-in logs

Post-auth, logs show a legitimate app (Office/Teams) signing in, reducing signal for defenders keying off app names.

The 15-minute problem → dynamic phishing

- ▶ device_code / user_code typically expires in ~15 minutes
- ▶ So actors coordinate in REAL TIME: build rapport first, then send the code only when the victim is live and engaged
 - Dynamic device code phishing: an attacker-controlled interstitial page generates a FRESH code on every visit
 - Russian threat actors(CozyLarch/UTA0304/UTA0307) - fake 'security check' page auto-mints codes; a watcher domain polls for success, then redirects to a real Teams meeting to look legitimate
 - Result: the phishing content never 'expires'

<https://www.volexity.com/blog/2025/02/13/multiple-russian-threat-actors-targeting-microsoft-device-code-authentication/>

Escalation: client-ID abuse → PRT

- ▶ The requested client_id shapes what the stolen token can do
- ▶ Storm-2372 shifted to the Microsoft Authentication Broker client_id
 - That refresh token can request a token for the device registration service → register an attacker-controlled device in Entra ID
 - With the new device identity + refresh token, obtain a Primary Refresh Token (PRT) → broad access to org resources
- ▶ Actors also used Office vs Teams client IDs; lateral movement via internal device-code lures from the victim's mailbox

Real-world campaigns (2024–2025)

Storm-2372

Suspected Russia-aligned; active since Aug 2024. Gov, NGO, IT, defense, telecom, health, energy across EU, N. America, Africa, Middle East. Teams-invite lures; broker client_id → PRT.

CozyLarch / APT29 (Volexity)

aka Midnight Blizzard. Spoofed US Dept. of State M365 external invitations and a fake Teams chat 'Measuring Influence Operations' via microsoft.com/devicelogin.

UTA0304 / UTA0307 (Volexity)

Signal → Element pivot impersonating Ukrainian MoD (UTA0304); EU Parliament / Trump themes with an auto-refreshing device-code page (UTA0307).

Lures & social engineering

- ▶ Impersonate a prominent, relevant person (officials, MoD, State Dept, MEPs) to build rapport first
- ▶ Move to encrypted chat (Signal, WhatsApp, Element) then send the code when the victim is live
- ▶ Themes: Teams/Element meeting invites, M365 external-tenant invitations, 'security check' interstitials
- ▶ Sender spoofing: mixed-encoding / Unicode 'friendly' From names to fake invites@microsoft.com (actually Gmail)
- ▶ Regionally-appropriate proxies (incl. US mobile networks) to blend in

Detection

```
high-signal indicators

// Entra ID sign-in logs
"authenticationProtocol": "deviceCode"           // the device-code sign-in
"originalTransferMethod": "deviceCodeFlow"       // on refreshed sessions

// Sequence heuristic: errorCode 50199 (prompt) THEN 0 (success)
//   within minutes = user paused to paste a phished code
```

- ▶ Monitor access to devicelogin / deviceauth URLs in email & proxy logs
- ▶ Watch Device Registration Service for new devices close-in-time to token activity (PRT abuse)

Prevention & mitigation

- ▶ Block the device code flow via Conditional Access (Authentication flows) wherever it isn't required — the single most effective control
- ▶ Scope any needed device-code usage to specific users/apps/compliant devices
- ▶ Prefer phishing-resistant MFA (FIDO2 / passkeys); avoid telephony MFA
- ▶ Restrict who can register devices in Entra ID (enrollment restrictions) to blunt PRT escalation
- ▶ On suspicion: revokeSignInSessions, force re-auth, apply sign-in risk policies + Continuous Access Evaluation
- ▶ User education: a legit app never asks you to enter a code someone sent you

Takeaways

The pattern

Abuse a legitimate OAuth flow so the victim completes the human half of the ATTACKER's login — tokens, not passwords, are phished.

Why it works

No malicious artifact, MFA is satisfied on real infra, and the session looks benign in logs.

The fix

Block device code flow in Conditional Access, use phishing-resistant MFA, and hunt deviceCode / deviceCodeFlow in sign-in logs.

Responsible disclosure & status

- ▶ Specific vulnerabilities were responsibly disclosed & patched
- ▶ Underlying patterns are NOT unique — similar weaknesses exist across other enterprise applications

Guidance for defenders

- ▶ Build hardened email-templating and notification pipelines
- ▶ Ensure your web services can't be hijacked to act as open relays
- ▶ Validate sender domains / enforce tenant boundaries on origination
- ▶ Audit HTML/subject injection points and input-length handling
- ▶ Detection: recognize behavioral signatures of these techniques

References

- ▶ Microsoft Threat Intelligence — 'Storm-2372 conducts device code phishing campaign' (Feb 13, 2025; upd. Feb 14, 2025)
- ▶ Volexity — 'Multiple Russian Threat Actors Targeting Microsoft Device Code Authentication' (Feb 13, 2025)
- ▶ RFC 8628 — OAuth 2.0 Device Authorization Grant
- ▶ Black Hills InfoSec — dynamic device code phishing
- ▶ Huntress — 'Oh-Auth: device code phishing in Google Cloud & Azure'
- ▶ Secureworks — family-of-client-ids research
- ▶ Entra CA guidance — block authentication flows / device code flow

Questions?

<https://linkedin.com/in/priyanknigam>

@Rev_Octo

Slides: <https://github.com/priyankn/Talks-Publications>